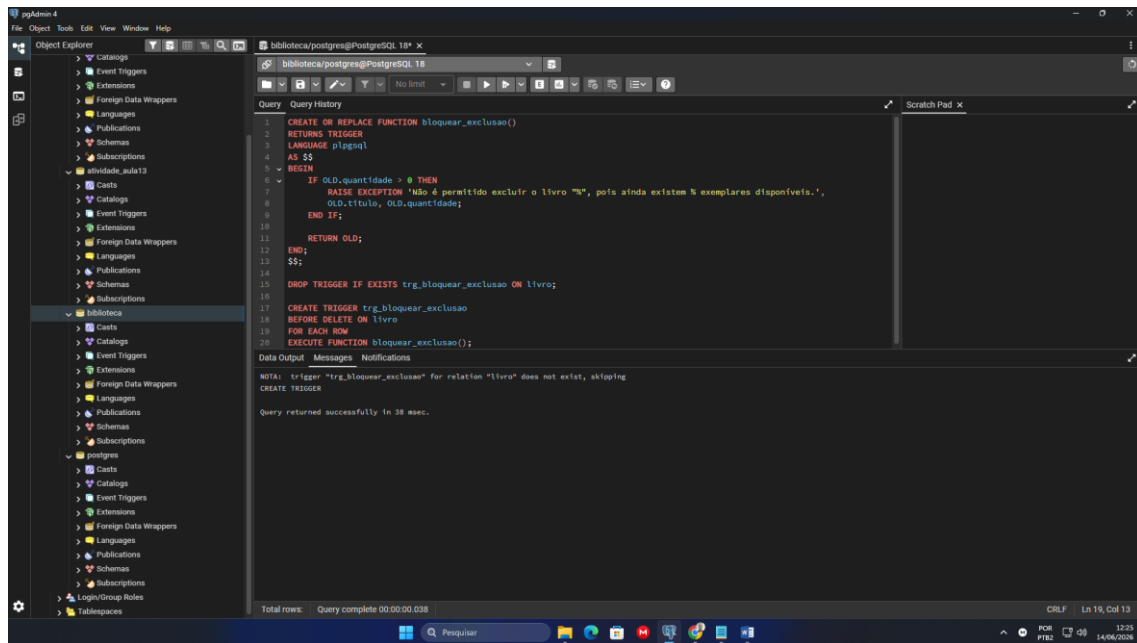


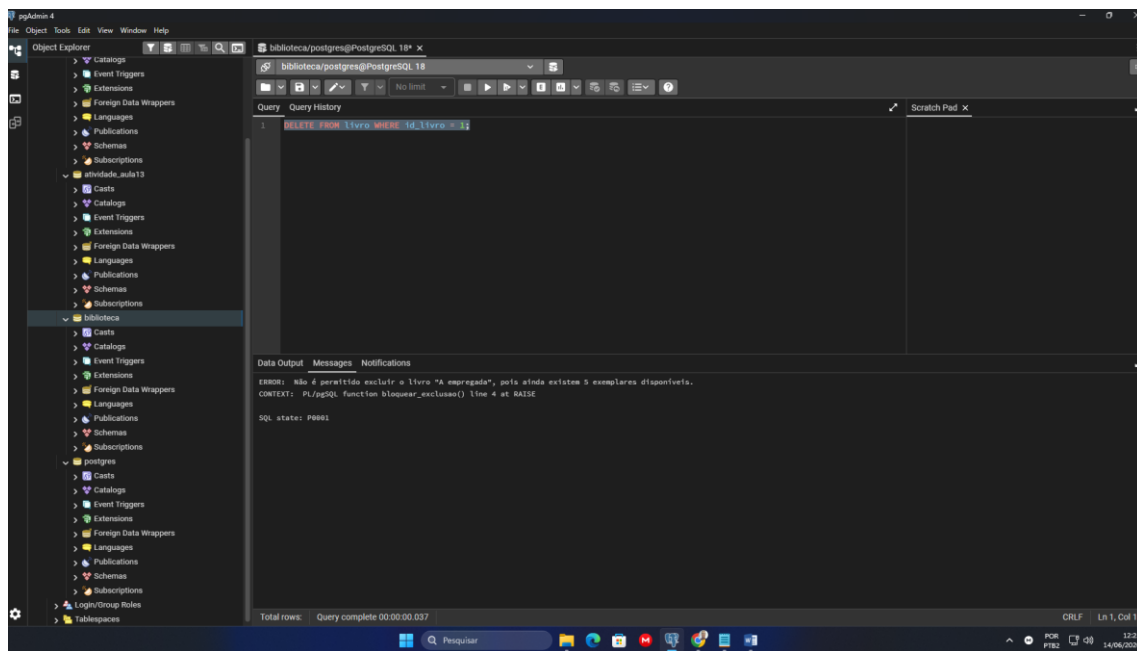
BDR-Atividade_Aula16-ThalesCambrãiaDias



The screenshot shows the pgAdmin 4 interface with the 'biblioteca/postgres@PostgreSQL 18' database selected. The 'Query' tab is active, displaying a SQL script that creates a function and a trigger. The script is as follows:

```
1 CREATE OR REPLACE FUNCTION bloquear_exclusao()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     IF OLD.quantidade > 0 THEN
7         RAISE EXCEPTION 'Não é permitido excluir o livro "%", pois ainda existem % exemplares disponíveis.',
8             OLD.titulo, OLD.quantidade;
9     END IF;
10    RETURN OLD;
11 END;
12 $$;
13 DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;
14 CREATE TRIGGER trg_bloquear_exclusao
15 BEFORE DELETE ON livro
16 FOR EACH ROW
17 EXECUTE FUNCTION bloquear_exclusao();
```

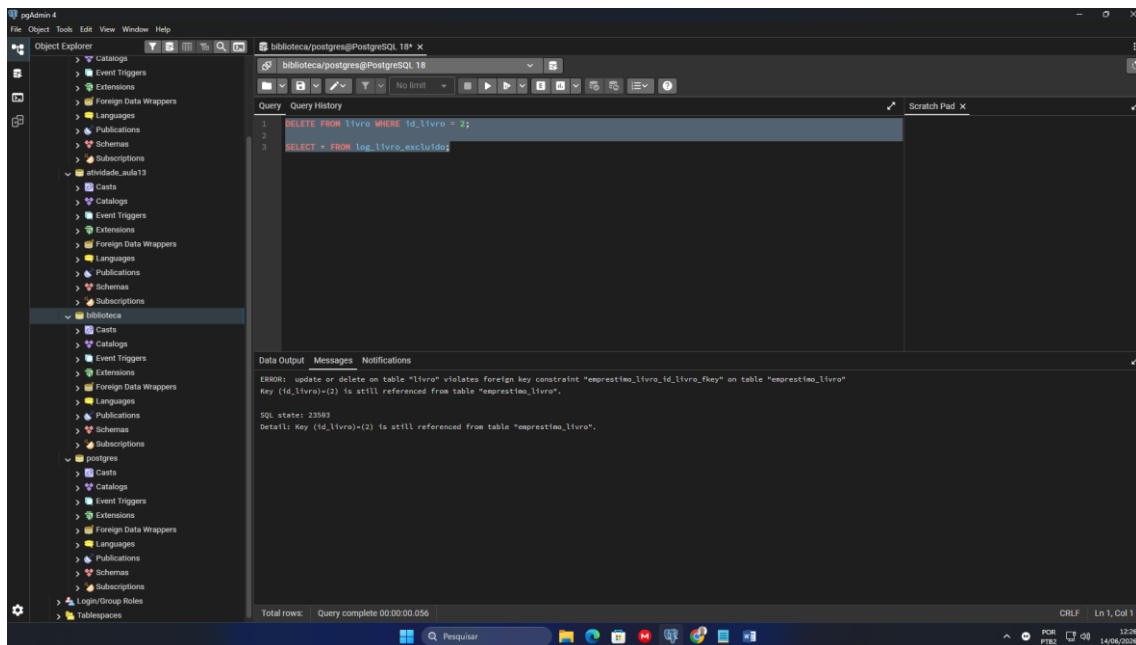
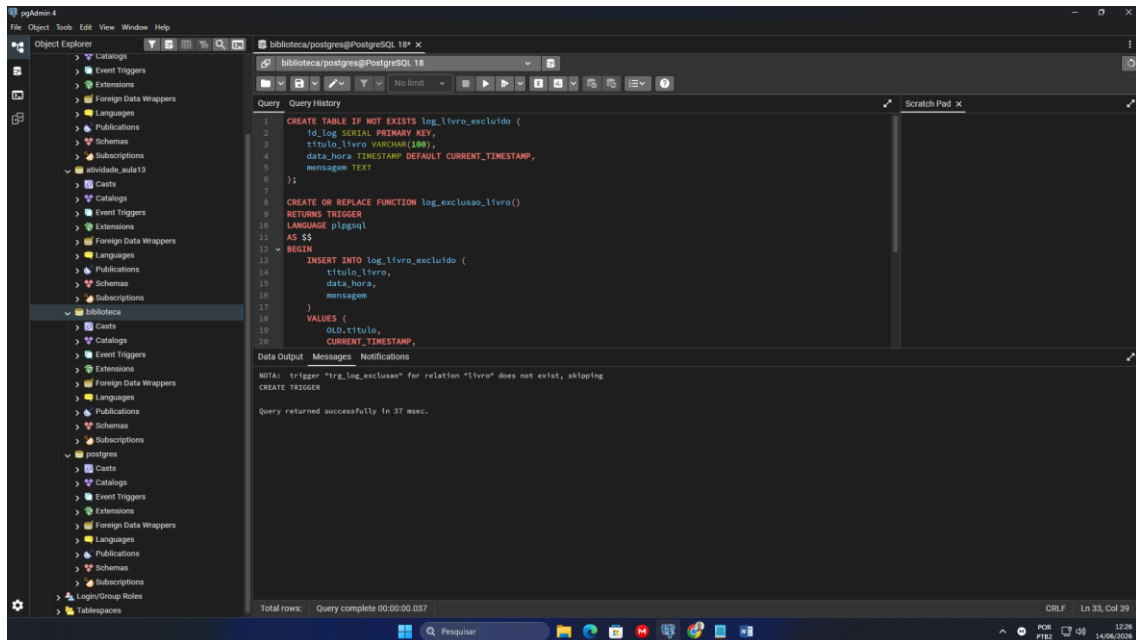
The 'Data Output' tab shows a message: 'NOTA: trigger "trg_bloquear_exclusao" for relation "livro" does not exist, skipping CREATE TRIGGER'. The 'Query History' tab shows the query completed successfully in 38 msec.

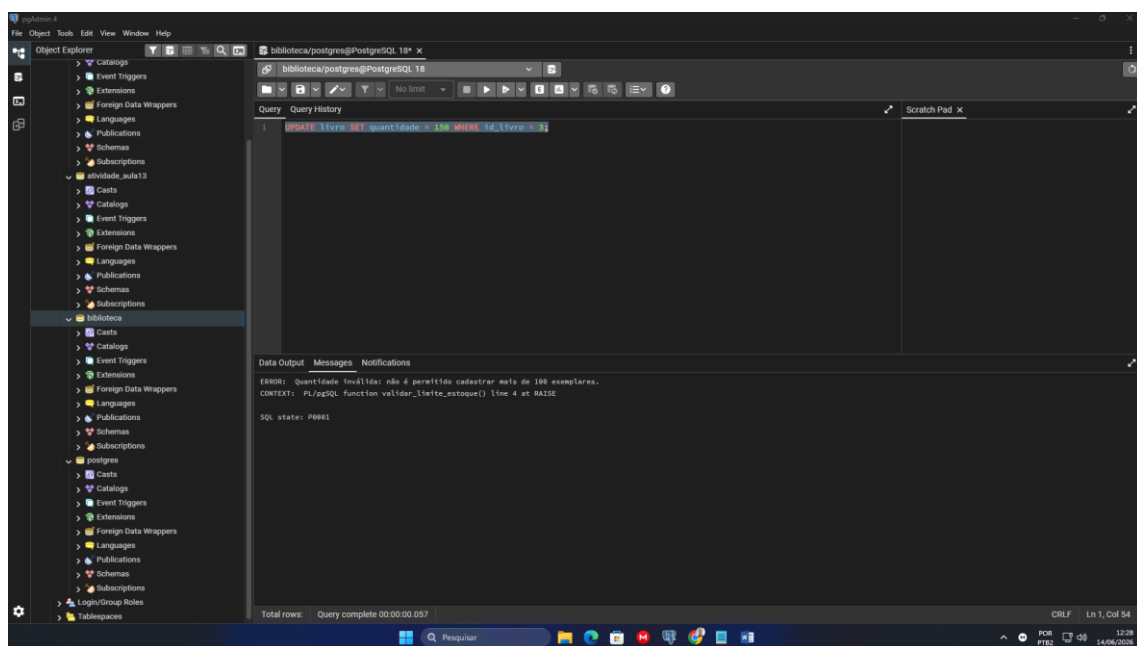
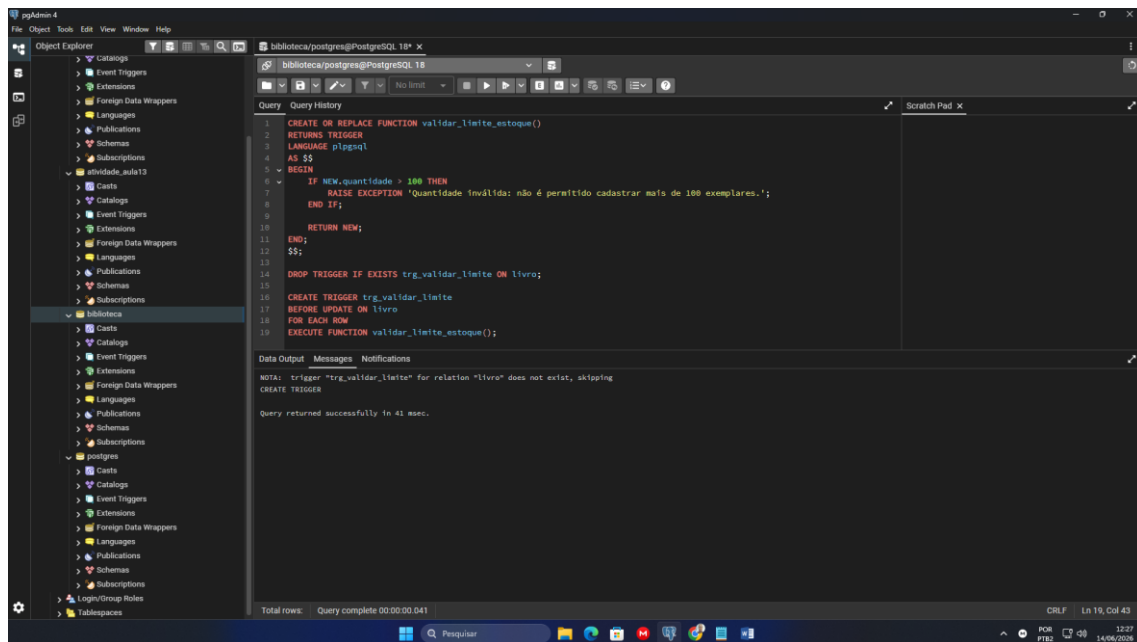


The screenshot shows the pgAdmin 4 interface with the 'biblioteca/postgres@PostgreSQL 18' database selected. The 'Query' tab is active, displaying a SQL script that attempts to delete a book. The script is as follows:

```
1 DELETE FROM livro WHERE id_livro = 1;
```

The 'Data Output' tab shows an error message: 'ERRO: não é permitido excluir o livro "A empregada", pois ainda existem 5 exemplares disponíveis. CONTEXT: PL/pgSQL function bloquear_exclusao() line 4 at RAISE'. The 'Query History' tab shows the query completed in 00:00:00.037.





Thales Cambraia Dias
Atividade Prática – Aula 16
Tema – Triggers

Exercício 1

```
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION 'Não é permitido excluir o livro "%", pois ainda existem % exemplares
disponíveis.',
            OLD.titulo, OLD.quantidade;
    END IF;

    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;

CREATE TRIGGER trg_bloquear_exclusao
BEFORE DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();
```

Teste bloqueando exclusão
DELETE FROM livro WHERE id_livro = 1;

Exercício 2

```
CREATE TABLE IF NOT EXISTS log_livro_excluido (
    id_log SERIAL PRIMARY KEY,
    titulo_livro VARCHAR(100),
    data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    mensagem TEXT
);

CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO log_livro_excluido (
        titulo_livro,
        data_hora,
        mensagem
    )
    VALUES (
```

```

        OLD.titulo,
        CURRENT_TIMESTAMP,
        'Livro removido do sistema com sucesso.'
    );

    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;

CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();

DELETE FROM livro WHERE id_livro = 2;
SELECT * FROM log_livro_excluido;

```

EXERCICIO 3

```

CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.quantidade > 100 THEN
        RAISE EXCEPTION 'Quantidade inválida: não é permitido cadastrar mais de 100
exemplares.';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validar_limite ON livro;

CREATE TRIGGER trg_validar_limite
BEFORE UPDATE ON livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();

```

Teste correto
UPDATE livro SET quantidade = 50 WHERE id_livro = 3;

Teste bloqueando atualização
UPDATE livro SET quantidade = 150 WHERE id_livro = 3;

Exercício 4

a) Trigger BEFORE é executada antes da operação acontecer.

Ela é usada para validar, alterar ou impedir a operação antes que ela seja gravada no banco.

Trigger AFTER é executada depois que a operação já aconteceu.

Ela é usada principalmente para auditoria, logs e registros históricos.

b) Para validação, deve ser usada trigger BEFORE, pois ela permite impedir a operação antes da gravação.

c) Para auditoria, deve ser usada trigger AFTER, pois ela registra algo que realmente aconteceu no banco.

d) A ordem é importante porque, em bancos reais, primeiro é necessário validar os dados.

Somente depois da operação concluída faz sentido registrar logs.

Se a auditoria acontecesse antes da validação, poderia registrar uma operação que foi bloqueada.

Exercício 5

a) Remover as triggers e deixar validações apenas na aplicação pode gerar riscos como: dados inconsistentes, exclusões indevidas, falta de auditoria e falhas caso outro sistema acesse o banco diretamente.

b) Manter regras no banco garante maior segurança, padronização e integridade, independentemente de qual aplicação esteja acessando os dados.

c) Triggers ajudam na integridade porque executam regras automaticamente em operações como INSERT, UPDATE e DELETE, evitando dados inválidos e registrando ações importantes.

d) Um exemplo real é um sistema financeiro que usa trigger para registrar automaticamente em uma tabela de auditoria toda alteração feita em pagamentos, valores ou dados de clientes.